

Mise à jour asynchrone des particules dans PPO-EDA

Thomas Landais

Juin 2026

1 Mise à jour partielle asynchrone

PPO-EDA maintient M particules $\theta \in \mathbb{R}^{B \times M \times N}$. En mode asynchrone, seules $\ell_{\text{active}} < M$ particules sont activées par itération, pour un budget d'évaluation de $\ell_{\text{active}} \times \lambda$ solutions (identique à un algorithme synchrone à ℓ_{active} agents).

1.1 Sélection UCB

Chaque particule est traitée comme un bras d'un bandit multi-bras [1]. Le score de la particule i à l'itération t est :

$$\text{UCB}_i(t) = \tilde{f}_i + c \sqrt{\frac{\ln(t+1)}{n_i+1}} \quad (1)$$

où $\tilde{f}_i \in [0, 1]$ est la fitness normalisée (baseline REINFORCE), n_i le nombre d'activations passées, et $c > 0$ le paramètre d'exploration (`ucb_c`). Les ℓ_{active} particules avec le score le plus élevé sont sélectionnées. Les particules jamais activées reçoivent un bonus 10^6 pour garantir leur exploration initiale.

En mode adaptatif (`ucb_adaptive`), c est mis à l'échelle par $\text{std}(\tilde{f})$, ce qui réduit l'exploration quand la population converge.

1.2 Mémoire de gradient

Un buffer `gradient_memory` $\in \mathbb{R}^{B \times M \times N}$ accumule le gradient REINFORCE de chaque particule :

$$g_i \leftarrow \nabla_{\theta_i} \mathcal{L}_{\text{REINFORCE}} \quad \text{uniquement si } i \in \mathcal{A}_t \quad (2)$$

Les gradients des particules inactives sont conservés d'une itération à l'autre.

1.3 SVGD partiel

Seules les particules actives sont mises à jour. Leur support SVGD inclut toutes les particules ayant été activées au moins une fois (`visited_mask`) :

$$\phi_i = \frac{1}{|\mathcal{S}_t|} \sum_{j \in \mathcal{S}_t} [k(\theta_j, \theta_i) g_j + \nabla_{\theta_j} k(\theta_j, \theta_i)], \quad \mathcal{S}_t = \text{visited_mask} \cup \mathcal{A}_t \quad (3)$$

puis $\theta_i \leftarrow \theta_i + \varepsilon \cdot \phi_i$ pour $i \in \mathcal{A}_t$.

2 Résultats

2.1 Exemple de table UCB

Le tableau 1 illustre l'état de la sélection UCB après $t = 3846$ itérations sur 10 particules ($M = 10$, mode adaptatif). La colonne Fit/N est la fitness moyenne normalisée par N , $Fit_norm = \tilde{f}_i \in [0, 1]$, $Explor = c\sqrt{\ln(t+1)/(n_i+1)}$, et $UCB = Fit_norm + Explor$. La particule 4 est la moins visitée (242 visites) et présente le score d'exploration le plus élevé, bien que sa fitness soit la plus faible.

TABLE 1 – Table UCB à l'instance 0, $t = 3846$, c adaptatif ($M = 10$).

Particule	Visites	Fit/ N	$\tilde{f}$	Explor	UCB
6	569	3.4087	1.0000	1.7396	2.7396
5	499	3.3269	0.9302	1.8574	2.7876
7	412	3.0529	0.6961	2.0437	2.7398
8	411	3.0865	0.7248	2.0462	2.7710
3	410	3.0529	0.6961	2.0487	2.7448
2	356	2.9038	0.5688	2.1982	2.7670
1	326	2.7933	0.4743	2.2968	2.7711
0	315	2.7212	0.4127	2.3364	2.7492
9	306	2.7115	0.4045	2.3704	2.7749
4	242	2.2380	0.0000	2.6644	2.6644

2.2 Comparaison Async vs Normal

Comparaison à budget constant ($\ell_{\text{active}} \times \lambda$ évaluations/itération) : async (M grande, ℓ_{active} actives, UCB) vs. normal ($M' = \ell_{\text{active}}$).

Références

- [1] P. Auer, N. Cesa-Bianchi, P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2) :235–256, 2002.

TABLE 2 – Score moyen (plus grand = meilleur). **TODO** : remplir après exécution de `main_expe_async_vs_normal.py`.

Instance	M	Score Async	Score Normal	Δ
NK dim64 $k=1$	–	–	–	–
NK dim128 $k=2$	–	–	–	–
NK dim256 $k=4$	–	–	–	–
NK3 dim64 $k=1$	–	–	–	–
NK3 dim128 $k=2$	–	–	–	–
QUBO dim64	–	–	–	–
QUBO dim128	–	–	–	–
Wins async / normal				– / –